How It Works

What was built, how every part works, and how to answer judges. Read this before you record.

Read first: the AI rule. CAC 2026 allows AI tools only if you fully disclose them, and AI "must not constitute the entirety of the technical development." Judges look for your own contribution and your understanding of the app. Right now, AI wrote almost all of this code. That is the biggest risk to your submission, bigger than any bug. Section 9 lists the specific work to do yourself before Oct 26, and gives you a disclosure draft.

Live link for judges: `https://ashih-10.github.io/Pepper/`. It runs as an on-device demo: no sign-up, nothing leaves the visitor's browser, and a pretend classmate (Jordan, group code `CHEM24`) shows how groups and deadline alerts work. Photo reading and the game plan give clearly labeled sample results there. The real app (accounts, live sync, real AI) runs once Supabase is set up (Setup Guide).

1. What's built

Everything on the PRD §10 "Build for the demo" list, plus several items the PRD deferred to the roadmap.

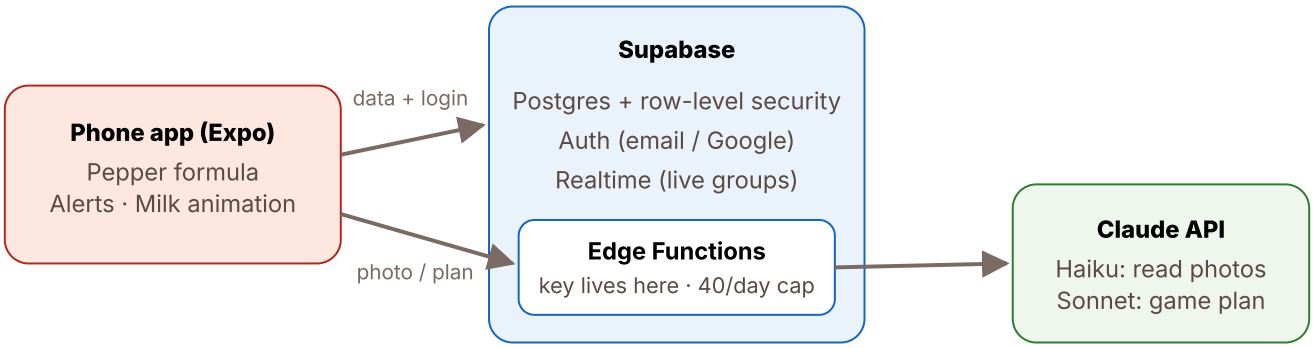
Feature	Status
Email sign-in (Google sign-in wired, needs your Google Cloud setup)	✅ Built
Onboarding: name, level, grade, classes + colors, goals & struggles, join a group	✅ Built (goals/struggles was a roadmap item)
Color-coded day strip + month calendar showing workload density	✅ Built
Task entry: type dropdown, custom types with a tier, optional grade-weight %	✅ Built
Screenshot-to-task: AI fills fields, flags unclear ones, asks a question, confirm at the bottom, handles multi-task syllabi	✅ Built
Pepper levels that heat up, personalized by workload	✅ Built
Pepper alerts by level, all hot tasks listed in each alert, quiet hours	✅ Built (local notifications)
Stacked-urgency callout ("4 urgent items, here's the order by weight")	✅ Built

Milk check-off animation + strikethrough history per day	✅ Built
Groups: create, join by code, share tasks, live sync, deadline-change alerts	✅ Built
Group activity feed, copy-on-delete, owner removes members, ownership hand-off	✅ Built (all were roadmap items)
Recurring tasks (daily / weekdays / weekly)	✅ Built (roadmap item)
AI "Today's game plan" using your onboarding answers	✅ Built (new; answers a PRD open question)
Demo tools: load a demo week, time-warp clock	✅ Built (for recording)

Decisions made on the PRD's open questions

Open question	What the app does
Checking off a shared task	Checks it off only for you (PRD's recommendation).
Who can edit a shared task	Any member. Deadline/type changes alert everyone. Only the creator or group owner can delete.
Invite link vs code	6-character code (no look-alike letters), with Copy and Share buttons. The owner can make a new code anytime, and removing a member changes it automatically.
Custom task weight	Student picks "counts like a Test / Quiz-Project / Homework".
Quiet hours	On by default, 10 PM–7 AM, adjustable. Anything urgent pings when quiet hours end.
COPPA	Minimum age 13, stated on sign-in. No parent-consent flow.
How AI uses onboarding answers	The game plan tailors its tips to your goals and struggles.
App name	"Pepper Planner" is a placeholder. Naming it is yours (Section 9).

2. How the pieces connect



The app talks to the database directly with the student's login token. **Row-level security** (rules inside the database) decides what each student can see, so even a modified app can't read someone else's tasks. The Claude API key lives only in Supabase's server secrets; the app never sees it.

3. The pepper formula

All in `src/lib/pepper.ts`. In one sentence: **heat = how much it matters × how little time you really have.**

1. **Weight (0–1).** Test 1.0, Quiz and Project 0.6, Homework 0.3. Custom types use the tier you pick. If you enter a grade %, that wins: 20%+ of the grade counts as much as a test.
2. **Real time left.** Days until due, minus the days of effort owed to *your other tasks due first* (test 1.5 days, quiz/project 0.75, homework 0.25), capped at 60% of the time left. This "crowding" is what makes the ranking personal.
3. **Urgency.** `1 ÷ (1 + real days left ÷ lead time)`. Lead time is how much runway a task needs: tests 4 days, quizzes 2.5, homework 2, projects 5 days or 35% of their timeline (whichever is longer). Urgency is 0.5 exactly one lead time out and 1.0 at the deadline.
4. **Neglect.** Projects get extra heat as their timeline passes while still unchecked (completion is one check-off, so unchecked means not started).
5. **Last-minute rule.** `heat = weight × urgency + (1 − weight) × urgency⁵`. The second part means even homework spikes in its final hours.
6. **Levels.** Heat 0–19 Bell · 20–34 Jalapeño · 35–54 Serrano · 55–74 Habanero · 75+ Ghost

Example (nothing else on your plate)	Heat	Level
Test in 21 / 10 / 5 / 3 / 1 days	16 / 29 / 44 / 57 / 80	Bell → Jalapeño → Serrano → Habanero → Ghost
Homework in 3 days / tomorrow night / tonight (8h) / 1 hour	13 / 24 / 58 / 93	Bell → Jalapeño → Habanero → Ghost

Project due in 10 days: added today vs. added 30 days ago	20 vs 58	Jalapeño vs Habanero (neglect)
Shared test, 4.7 days out: busy student (homework, quiz, test first) vs. empty week	**64 vs 46**	**Habanero vs Serrano**: same task, different heat

After the due date: tests and quizzes go quiet (nothing left to do). Homework, projects and custom tasks stay at Ghost until checked off, since they can still be turned in late.

> **Why a formula and not AI for the ranking?** The PRD said "AI ranks every task." Asking an AI model to rank every task every minute would be slow, cost money on every recalculation, fail without internet, and give slightly different answers each time, so peppers would flicker. The formula is instant, free, works offline, gives the same answer every time, and you can explain it step by step. AI is used where it is actually better: reading messy photos and writing personal advice. Judges tend to respect this kind of tradeoff.

4. Where AI is used (and the guardrails)

	Screenshot-to-task	Today's game plan
Model	Claude Haiku 5.5 (fast, cheap, reads images)	Claude Sonnet 5.5 (better judgment)
Input	Photo (resized on the phone), today's date, your class names	Your hot tasks *already ranked by the formula* + your onboarding answers
Output	Structured JSON (schema-enforced): title, class, date, time, type, weight, which fields were unclear, one question for you	One tip and a time block per task, in the formula's order
Guardrails	Server re-checks everything: impossible dates dropped, class names matched to yours, unclear fields highlighted ⚠️, confirm button at the bottom. Text in the photo is treated as data, not instructions.	Can't reorder or invent tasks: the server forces the formula's order and drops unknown IDs. A safety fallback is enabled: if the model declines a request, the API retries on a fallback model in the same call.

Both functions check your login, reject malformed requests before calling Claude, and enforce a cap of 40 AI calls per student per day.

5. Security and privacy

Data	Who can see it
Personal tasks, classes	Only you
Whether you finished a shared task, your pepper levels	Only you (pepper levels are computed on your phone)

Shared task title / date / type, activity feed, roster	Members of that group
Your name	You and people in your groups
Your goals, struggles, grade and quiet hours	Only you (groupmates see your name and nothing else)

- Group create/join/leave happen through server functions, so nobody can join without the code or make themselves owner.
- Column-level locks: students can't change who created a task, their role, or the activity feed.
- Wrong join codes are limited to 20 tries a day per student, so codes can't be guessed by brute force.
- Only the person who added a shared task can move it to another group; only whoever created a repeating task can add to it or stop it.
- 27 automated tests sign in as several students (owner, member, outsider) and try to break these rules. All pass.
- A multi-agent security red-team attacked the backend in three rounds, as fake students against a throwaway copy of the database. Nobody could read another student's private data or take over an account or group. Every confirmed issue was fixed and re-attacked. Details are in the **Security Review** PDF.

6. Data model

Table	Holds
`profiles`	Name, level, grade, goals, struggles, quiet hours
`classes`	Your classes, colors and order
`tasks`	Personal and shared tasks (shared = has a group)
`task_series`	Repeating tasks: the rule, the first due date, and how far ahead repeats exist
`task_completions`	Who checked off what, and when (private)
`groups`, `group_members`	Groups, join codes, owner, each member's class color link
`activity`	Group feed, written automatically by database triggers
`task_rescues`	"Keep a copy?" offers when a shared task or group is deleted
`ai_usage`	Daily AI call counter per student

7. How it was tested

Test	Result

Unit tests: pepper formula (incl. your demo scenario), recurring tasks, alert planning, the on-device demo backend	58 / 58 pass
Security tests on a real local Supabase (students trying to break the rules)	27 / 27 pass
AI functions end to end against a fake Claude server (login, bad input, request format, safety fallback, daily cap, bad requests don't use up the cap, oversized uploads)	7 / 7 pass
Full demo script in a real browser with two accounts, plus stop-repeating and new invite code (screenshots below)	Pass, 0 console errors
Automated multi-agent code review (security, logic, app state, screens)	41 confirmed issues (5 serious), all fixed and re-tested
Security red-team: agents attacking as fake students, three rounds, every finding independently reproduced	No data leaks or takeovers; all confirmed issues fixed (Security Review PDF)
Demo mode in a real browser: network traffic recorded for the whole flow	0 requests leave the device
Type-check, lint, Expo doctor (21 checks), iOS + Android bundles	Clean

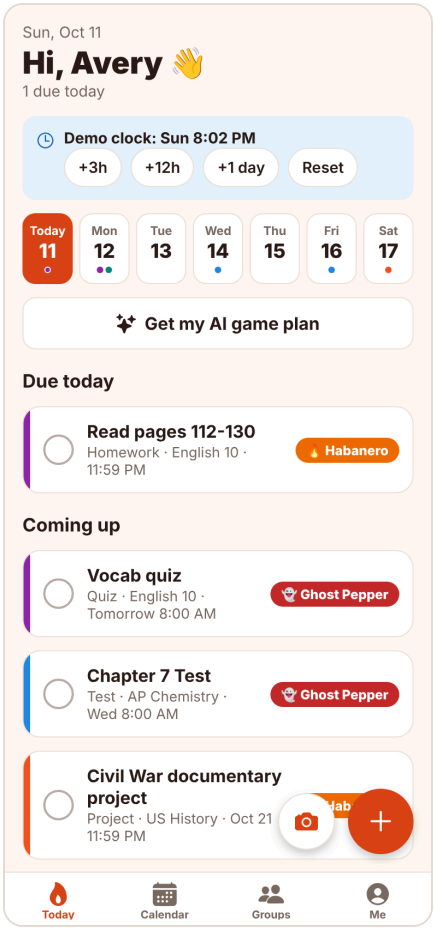
Not yet tested (be honest about this if asked):

- **Real Claude calls.** No API key was available, so the AI was tested against a fake server that mimics Claude's exact format. Your first real photo is the first real call.
- **On an actual phone:** camera, haptics and notifications. The code compiles for iOS and Android, but only the web version was run.
- **Google sign-in** (needs your Google Cloud setup).

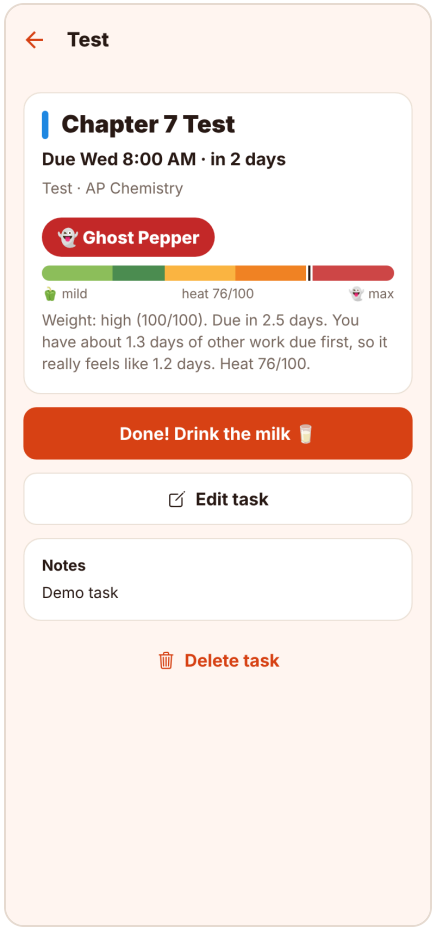
Known limits of the MVP

- Alerts are scheduled on the phone each time the app opens or a task changes. They fire with the app closed, but a change made by a groupmate reaches you when you next open the app, not as a push while it's closed. Server push is on the roadmap.
- Phones cap queued notifications (iOS: 64), so Ghost Pepper's 10-minute alerts cover about the next 10 hours, and the queue refills every time the app opens.
- Deleting one repeat of a repeating task deletes only that one. To end the whole thing, open it and tap **Stop repeating** (only whoever created it can).

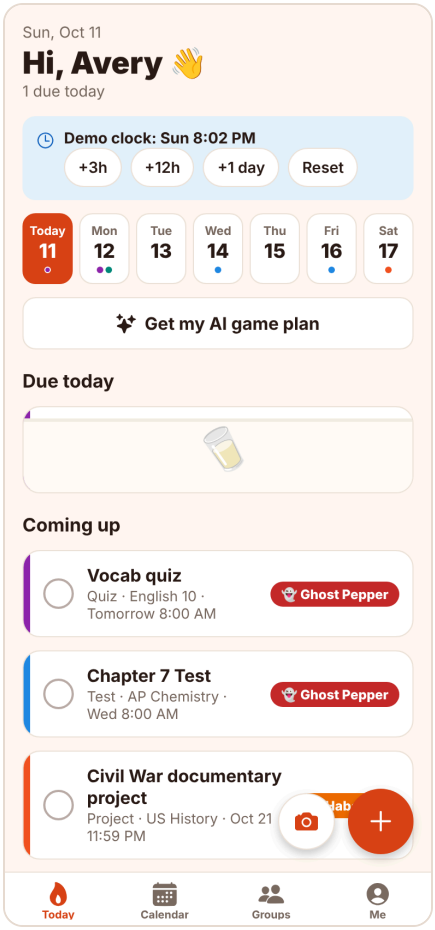
8. Screenshots from the test run



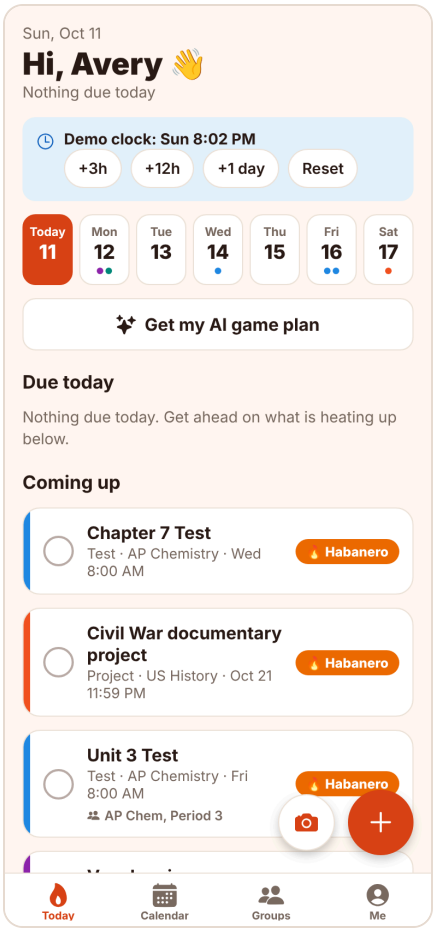
Today: the demo week, hottest first



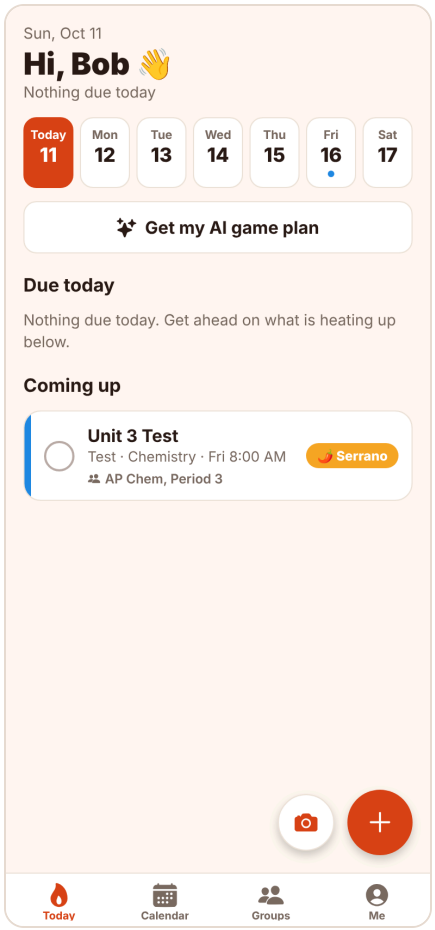
Task detail: heat bar + plain-English "why"



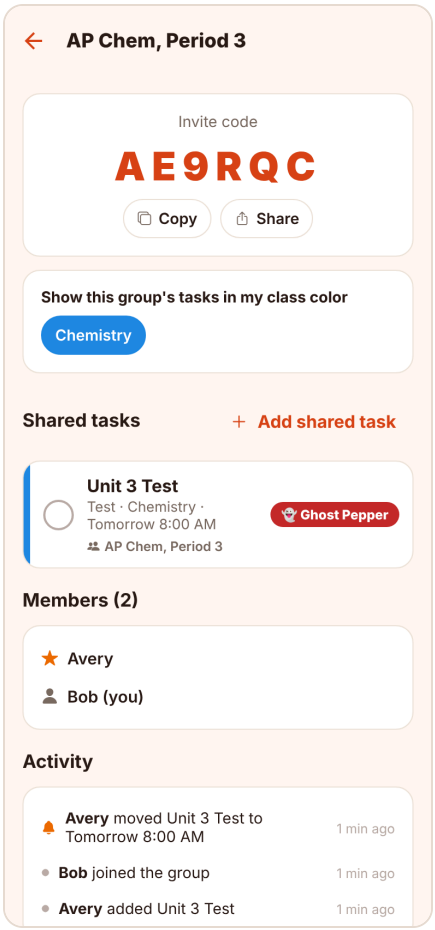
Check-off: the milk mid-pour



Busy student: shared Unit 3 Test is 🔥 Habanero



Same test, empty week: 🌶️ Serrano



Group page: code, roster, live activity feed

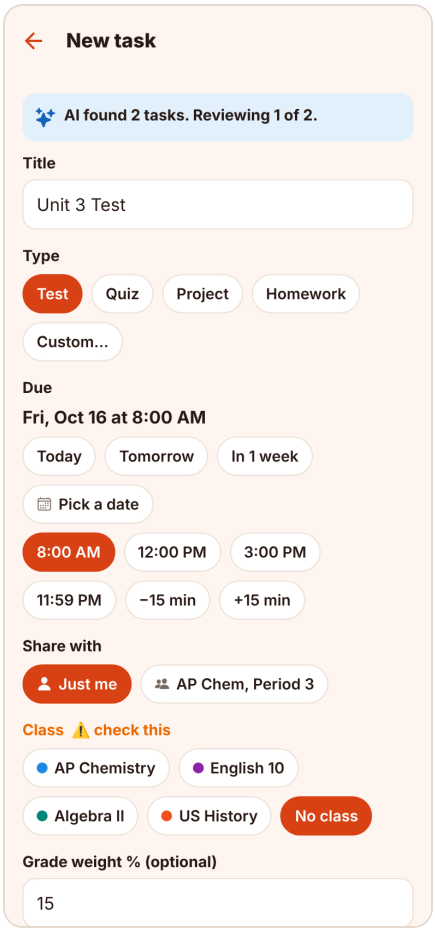
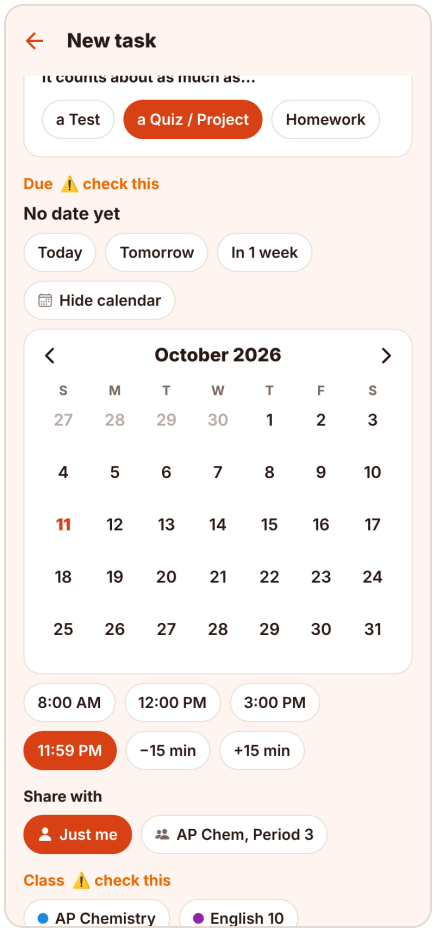
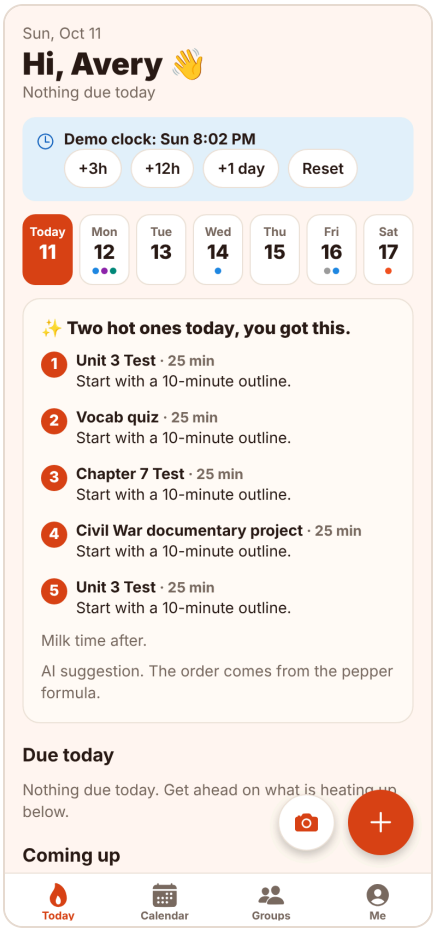


Photo → task: AI-filled form



Unclear fields flagged ⚠️ for the student



Game plan (fake-AI text in this test run)

9. Your part before Oct 26 (do not skip)

To meet the CAC rule, you need real, explainable work of your own in the app. Pick at least the first three. Use AI as a tutor that explains, not as the one typing.

1 · Name, icon, splash
Name the app (it's still "Pepper Planner") and design the icon and splash screen. Change `name` in `app.json` and the images in `assets/images`. ~1–2 hours.

2 · Tune the formula
Use it for a week with your real schedule. Change `LEAD_DAYS`, `PREP_DAYS` or `THRESHOLDS` in `pepper.ts` until it feels right, and write one new test in `tests/pepper.test.ts` for your case. ~2 hours.

3 · Build one feature yourself
Best pick, because it's your story: **practice/class quiet hours**. Weekly blocks (e.g. Mon–Fri 3:30–6 PM practice) when alerts hold. It touches `alert-plan.ts` + the Me screen. ~4–6 hours.

Also: be able to explain every file in the "Where things live" list in the README; test with 2–3 classmates and fix what confuses them; record the video yourself.

Disclosure draft (edit the brackets so it's 100% true)

> "I used Claude Code, an AI coding assistant, to help build this app. I wrote the product requirements: the features, the pepper urgency system, how groups share tasks, and the demo plan. Claude Code generated most of the initial code, database and tests from my spec. I then [designed the name and icon], [tuned the urgency formula after using the app for a week and added my own test], and [built the practice quiet-hours feature myself]. Inside the app, AI is used in two places: Claude Haiku reads photos of assignment sheets, and Claude Sonnet writes a daily study plan. The urgency ranking itself is a formula I designed and can explain step by step."

10. Questions judges will likely ask

Why not let AI rank the tasks?

Speed, cost, offline use, consistency, and it's explainable. AI is used for the two jobs a formula can't do: reading messy photos and giving personal advice.

How can the same shared test have different pepper levels for two students?

Crowding. Effort owed to your other tasks due first is subtracted from the time you have left. Three tests that week means less real time, so more heat.

What if the AI misreads a photo?

It marks fields it's unsure of, asks a question, and the confirm button is at the bottom of the form, so you scroll past every field. The server also throws out impossible dates and classes that aren't yours.

How do you stop students seeing each other's data?

Row-level security in the database, tested by 27 automated attack tests and a three-round security red-team. Completion status, pepper levels and onboarding answers are private.

Couldn't someone steal your API key from the app?

It isn't in the app. It lives in Supabase server secrets, with a 40-calls-a-day cap per student and a spending limit on the account.

Won't constant alerts make people ignore it?

Alerts scale with heat (one quiet ping for Jalapeño up to every 10 minutes for Ghost), each alert lists all hot tasks in order, quiet hours stop alerts at night, and group alerts only fire for date or type changes.

Why do tests stop alerting after the date, but homework doesn't?

You can't take a test late, but you can usually turn homework in late, so it stays at max heat until it's checked off.

What happens if a group owner deletes the group?

Every other member is offered a copy of each unfinished task first.

How did you test it?

92 automated tests plus a full two-account run of the demo script (Section 7). An automated code review found 41 issues and a security red-team found more; all were fixed and re-tested, and the fixes were attacked again.

What's next?

Server push notifications, alarm sounds for Ghost Pepper, schedule-aware quiet hours, Google Classroom/Canvas import, and offline mode.